

Session 9

Graphs

Scope

2

Graphs:

- Directed and undirected graphs
- Weighted graphs (networks)
- Common graph algorithms

Graphs

3

Like trees, *graphs* are made up of nodes and the connections between those nodes

In graph terminology, we refer to the nodes as *vertices* and refer to the connections among them as *edges*

Vertices are typically referenced by a name or label

Edges are referenced by a pairing of the vertices (A, B) that they connect

An *undirected graph* is a graph where the pairings representing the edges are unordered

Undirected Graphs

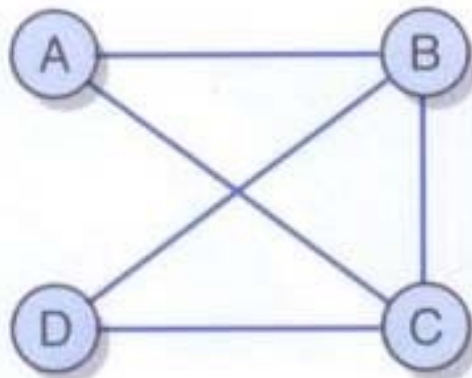
4

An edge in an undirected graph can be traversed in either direction

Two vertices are said to be *adjacent* if there is an edge connecting them

Adjacent vertices are sometimes referred to as *neighbors*

An undirected graph is considered *complete* if it has the maximum number of edges connecting vertices



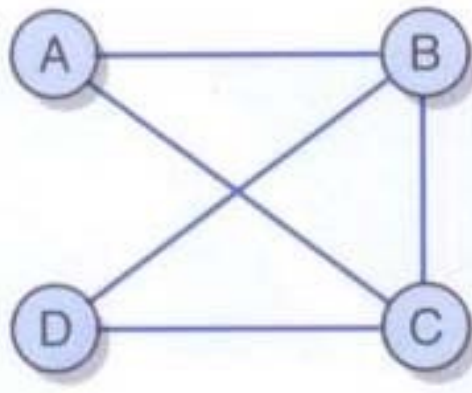
Undirected Graphs

5

A *path* is a sequence of edges that connects two vertices in a graph

The *length* of a path in is the number of edges in the path (or the number of vertices minus 1)

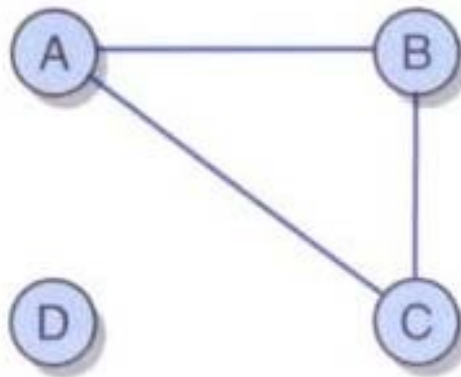
An undirected graph is considered *connected* if for any two vertices in the graph there is a path between them



Undirected Graphs

6

An example of an undirected graph that is not connected:



Cycles

7

A *cycle* is a path in which the first and last vertices are the same and none of the edges are repeated

A graph that has no cycles is called *acyclic*

Directed Graphs

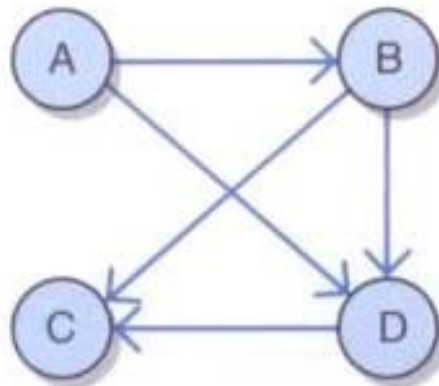
8

A *directed graph*, sometimes referred to as a *digraph*, is a graph where the edges that only flow in one direction

This means that the edges (A, B) and (B, A) are separate, directional edges in a directed graph

A directed graph with

- vertices A, B, C, D
- edges (A, B) , (A, D) , (B, C) , (B, D) and (D, C)



Directed Graphs

9

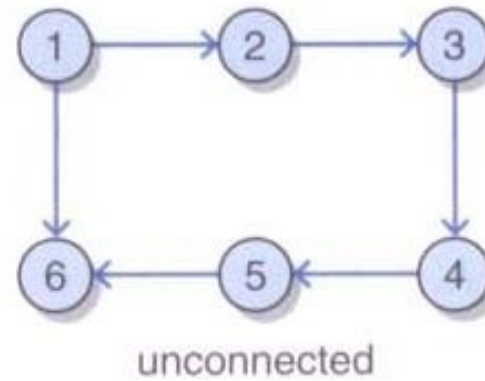
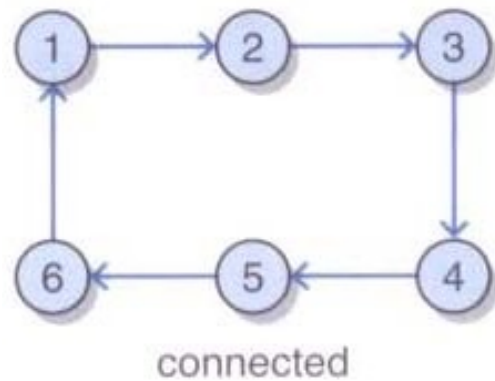
Previous definitions change slightly for directed graphs

- a path in a direct graph is a sequence of directed edges that connects two vertices in a graph
- a directed graph is connected if for any two vertices in the graph there is a path between them
- if a directed graph has no cycles, it is possible to arrange the vertices such that vertex A precedes vertex B if an edge exists from A to B

Directed Graphs

10

A connected directed graph and an unconnected directed graph:



Weighted Graphs

11

A *weighted graph* is a graph with weights (or costs) associated with each edge

The weight of a path in a weighted graph is the sum of the weights of the edges in the path

Weighted graphs may be either undirected or directed

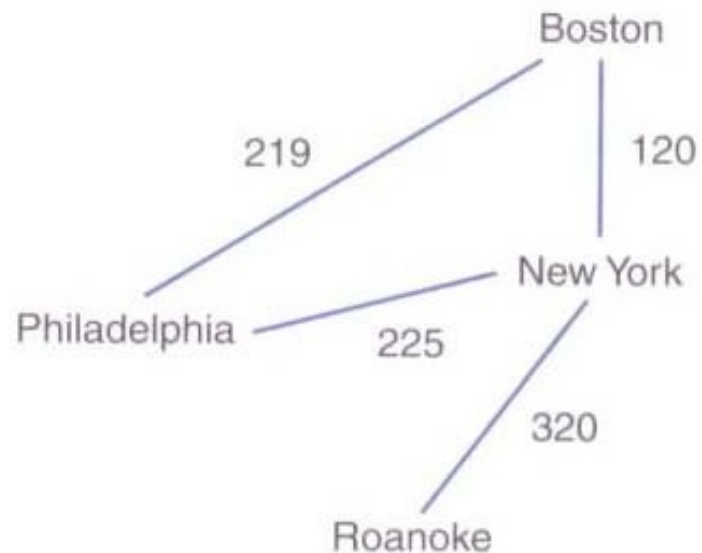
For weighted graphs, we represent each edge with a triple including the starting vertex, ending vertex, and the weight (Boston, New York, 120)

Weighted Graphs

12

We could use an undirected network to represent flights between cities

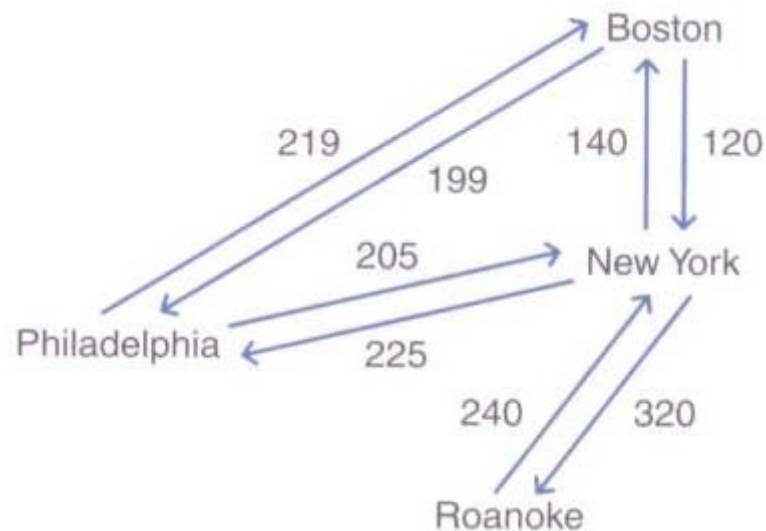
The weights are the cost



Weighted Graphs

13

A directed version of the graph could show different costs depending on the direction



Common Graph Algorithms

14

There are a number of a common algorithms that may apply to undirected, directed, and/or weighted graphs

These include

- various traversal algorithms
- algorithms for finding the shortest path
- algorithms for finding the least costly path in a network
- algorithms for answering simple questions (such as connectivity)

Graph Traversals

15

There are two main types of graph traversal algorithms

- *breadth-first*: behaves much like a level-order traversal of a tree
- *depth-first*: behaves much like the preorder traversal of a tree

One difference: there is no root node present in a graph

Graph traversals may start at any vertex in the graph

Breadth-First Traversal

16

We can construct a breadth-first traversal for a graph using a queue and an iterator

We will use the queue to manage the traversal and the iterator to build the result

Breadth-first traversal algorithm overview

- enqueue the starting vertex, and mark it as “visited”
- loop until queue is empty
 - dequeue first vertex and add it to iterator
 - enqueue each unmarked vertex adjacent to the dequeued vertex
 - mark each vertex enqueued as “visited”

Breadth-First Traversal

17

```
public Iterator<T> iteratorBFS(int startIndex)
{
    Integer x;
    QueueADT<Integer> traversalQueue = new LinkedQueue<Integer>();
    UnorderedListADT<T> resultList = new ArrayUnorderedList<T>();

    if (!indexIsValid(startIndex))    return resultList.iterator();

    boolean[] visited = new boolean[numVertices];
    for (int i = 0; i < numVertices; i++)
        visited[i] = false;

    traversalQueue.enqueue(new Integer(startIndex));
    visited[startIndex] = true;
    while (!traversalQueue.isEmpty())
    {
        x = traversalQueue.dequeue();
        resultList.addToRear(vertices[x.intValue()]);

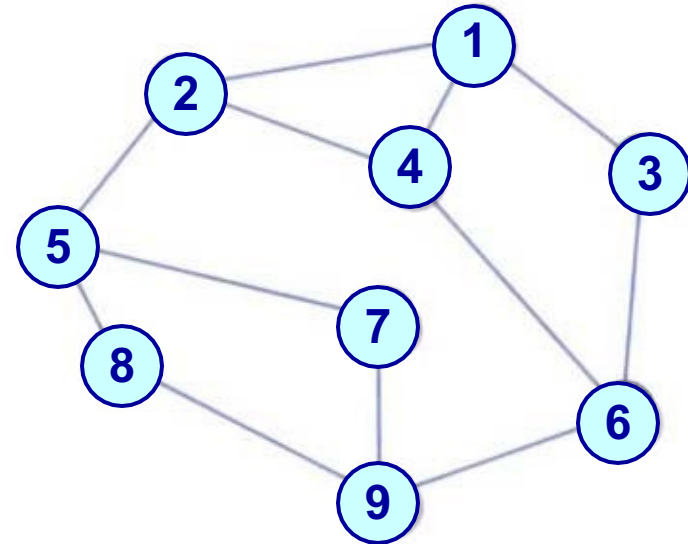
        // Find all vertices adjacent to x that have not been visited
        // and queue them up

        for (int i = 0; i < numVertices; i++)
        {
            if (adjMatrix[x.intValue()][i] && !visited[i])
            {
                traversalQueue.enqueue(new Integer(i));
                visited[i] = true;
            }
        }
    }
    return new GraphIterator(resultList.iterator());
}
```

Graph Traversals

18

A Breadth-First Traversal example:



`startIndex = 5`

`visited`

T T T T T T T T T

`traversalQueue`

5 7 8 1 4 9 3 6

Done!

`resultList`

Depth-First Traversal

19

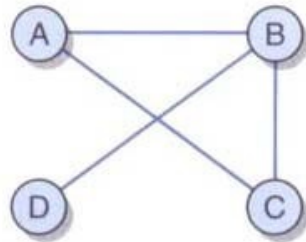
Use nearly the same approach as the breadth-first traversal, but replace the queue with a stack

Additionally, do not mark a vertex as visited until it has been added to the iterator

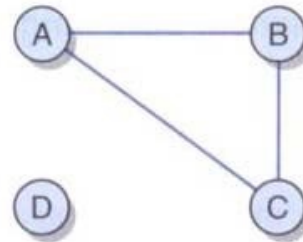
Connectivity

20

A graph is connected if and only if for each vertex v in a graph containing n vertices, the size of the result of a breadth-first traversal at v is n



Starting Vertex	Breadth-First Traversal
A	A, B, C, D
B	B, A, D, C
C	C, B, A, D
D	D, B, A, C



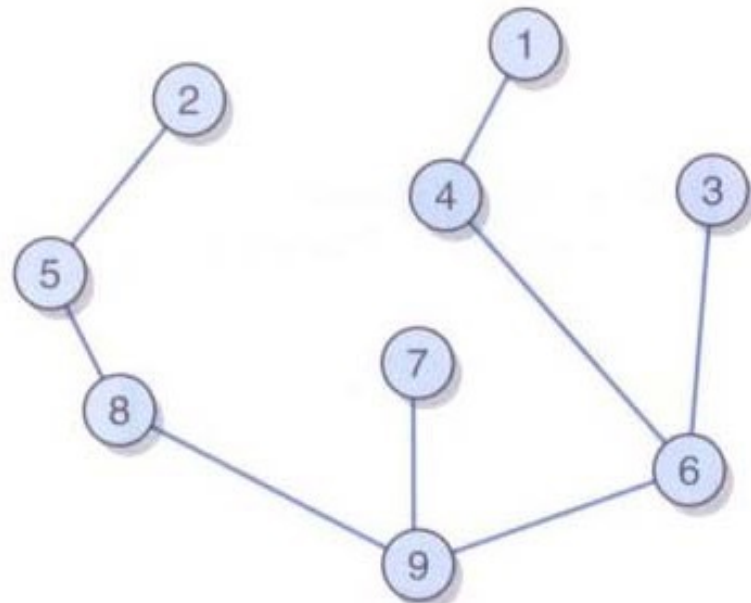
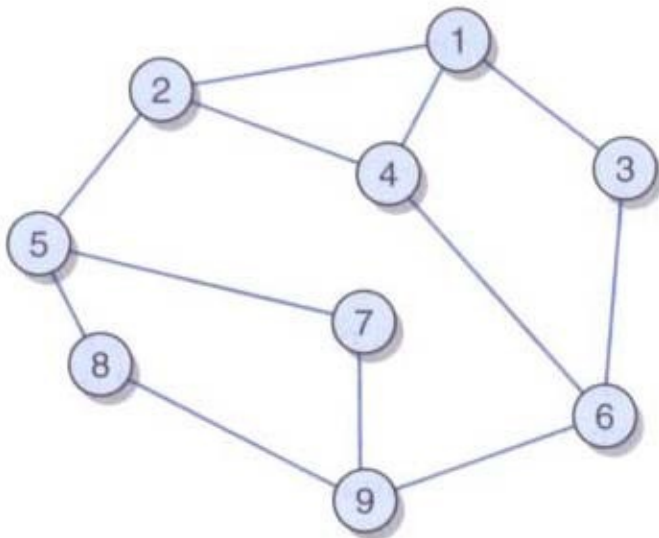
Starting Vertex	Breadth-First Traversal
A	A, B, C
B	B, A, C
C	C, B, A
D	D

Spanning Trees

21

A *spanning tree* is a tree that includes all of the vertices of a graph and some, but possibly not all, of the edges

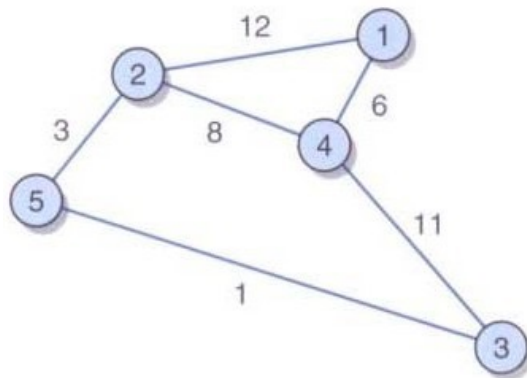
Since trees are also graphs, for some graphs, the graph itself will be a spanning tree, and thus the only spanning tree



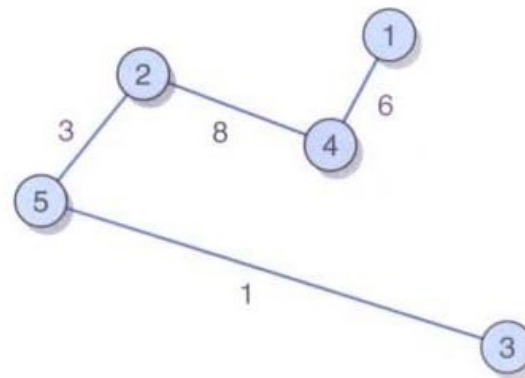
Minimum Spanning Tree

22

A *minimum spanning tree* (MST) is a spanning tree where the sum of the weights of the edges is less than or equal to the sum of the weights for any other spanning tree for the same graph



Network



Minimum Spanning Tree

Minimum Spanning Tree

23

Minimal Spanning Tree algorithm overview

- pick an arbitrary starting vertex and add it to our MST
- add all of the edges that include our starting vertex to a minheap ordered by weight
- remove the minimum edge from the minheap and add the edge and the new vertex to our MST
- add to the minheap all of the edges that include this new vertex and whose other vertex is not already in the MST
- continue until the minheap is empty or all vertices are in the MST

Shortest Path

24

There are two possibilities for determining the “shortest” path in a graph

- determine the literal shortest path between a starting vertex and a target vertex (the least number of edges between the two vertices)
 - simplest approach – a variation of the breadth-first traversal algorithm
- look for the cheapest path in a weighted graph
 - use a minheap or a priority queue storing vertex, weight pairs

Implementing Graphs

25

Strategies for implementing graphs:

Adjacency lists

- use a set of graph nodes which contain a linked list storing the edges within each node
- for weighted graphs, each edge would be stored as a triple including the weight

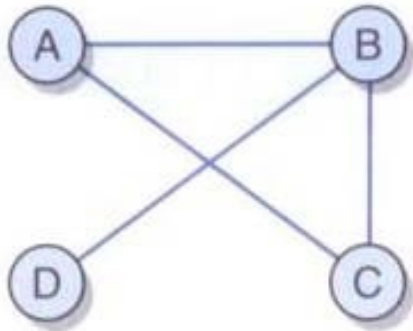
Adjacency matrices

- use a two dimensional array
- each position of the array represents an intersection between two vertices in the graph
- each intersection is represented by a boolean value indicating whether or not the two vertices are connected

Adjacency Matrices

26

An undirected graph and it's adjacency matrix:

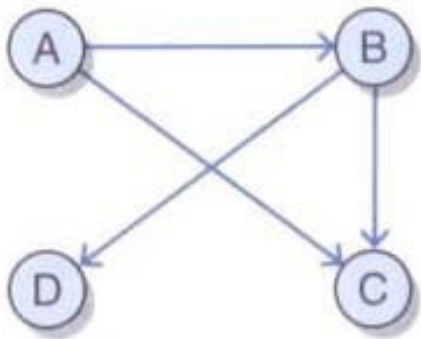


	A	B	C	D
A	F	T	T	F
B	T	F	T	T
C	T	T	F	F
D	F	T	F	F

Adjacency Matrices

27

A directed graph and its adjacency matrix:



	A	B	C	D
A	F	T	T	F
B	F	F	T	T
C	F	F	F	F
D	F	F	F	F

GraphADT

28

```
public interface GraphADT<T>
{
    // Adds a vertex to this graph, associating object with vertex.
    public void addVertex(T vertex);

    // Removes a single vertex with the given value from this graph.
    public void removeVertex(T vertex);

    // Inserts an edge between two vertices of this graph.
    public void addEdge(T vertex1, T vertex2);

    // Removes an edge between two vertices of this graph.
    public void removeEdge(T vertex1, T vertex2);

    // Returns a breadth first iterator starting with the given vertex.
    public Iterator iteratorBFS(T startVertex);

    // Returns a depth first iterator starting with the given vertex.
    public Iterator iteratorDFS(T startVertex);

    // Returns an iterator that contains the shortest path between two vertices.
    public Iterator iteratorShortestPath(T startVertex, T targetVertex);
}
```

Key Things to take away:

29

Graphs:

- An undirected graph has edges which may be traversed in both directions
- A directed graph has edges which may only be traversed in one direction
- A Path is a sequence of edges from one vertex in the graph to another
- A Cycle is a path in which the first vertex is the same as the last